

**CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
(CHARUSAT)**

Department of Computer Engineering

SOFTWARE REQUIREMENTS SPECIFICATION

PlaceIntel

Online Placement Intelligence Platform

Semester: 5th, B.Tech Computer Engineering

Academic Year: 2025–2026

FINAL REVISED VERSION

Document Version: 1.0

Prepared By

24DCE043 (Om Ghorl)

24DCE060 (Harsh Korat)

24DCE067 (Het Mangukiya)

ABSTRACT

PlaceIntel is a web-based placement intelligence platform for CHARUSAT. We designed it around a simple difficulty students face during placement preparation: the useful information is already present, but it is spread across Google Groups emails, PDFs, and forwarded messages. In this version, an administrator enters each real placement notice once. Students can then search and filter a structured collection of companies and opportunities, inspect eligibility and package details, view trends on an analytics dashboard, and see a personal Fit Score on each listing. The system also includes an AI chatbot built in the same semester. It retrieves relevant chunks from the placement notices stored in PostgreSQL with pgvector and asks an LLM to answer from those chunks, while naming the source notice and admitting when the available data does not contain an answer. The platform uses React and TypeScript on the frontend, Node.js and Express on the backend, Prisma for database access, PostgreSQL 15 for storage, JWT and bcrypt for authentication, and Docker Compose for local development. This report records the requirements, design choices, development plan, data model, interfaces, testing considerations, and the boundaries of this version.

TABLE OF CONTENT

ABSTRACT.....	2
TABLE OF CONTENT	3
LIST OF FIGURES	5
LIST OF TABLES	5
1. INTRODUCTION	7
1.1 Problem Summary	7
1.1.1 Problem Identification	7
1.1.2 Problem Solution	7
1.2 What's Different About This Project.....	8
1.2.1 The Fit Score.....	8
1.2.2 Building the Chatbot.....	8
1.3 Purpose of This Document.....	8
1.4 Scope and Boundaries.....	8
1.5 Stakeholders and Intended Users	9
1.6 Assumptions and Dependencies	9
1.7 Document Overview	9
2. PLANNING	10
2.1 Model Description — Iterative Waterfall Model.....	10
2.2 Our Schedule.....	11
2.3 Risk Management	12
2.3.1 Risk Identification.....	12
2.3.2 Risk Containment.....	13
3. DETAIL DESCRIPTION	14
3.1 System Overview	14
3.2 Functional Areas	15
3.2.1 Authentication and Roles.....	15
3.2.2 Placement Search and Filters	15
3.2.3 Fit Score	15

3.2.4 Analytics Dashboard.....	16
3.2.5 AI Chatbot.....	16
3.3 API Endpoint Summary	17
3.4 Validation and Error Handling.....	18
3.5 Non-Functional Targets	18
3.6 Practical Acceptance Checks	19
4. DIAGRAMS	20
4.1 High Level Architecture	20
4.2 Use Case Diagram.....	21
4.3 Student Activity Diagram	22
4.4 Student Sequence Flow Diagram	23
4.5 Data Flow Diagram Level – 0.....	24
4.6 Data Flow Diagram Level – 1.....	25
5. DATA DICTIONARY.....	26
5.1 Company	26
5.2 Placement.....	27
5.3 Skill	28
5.4 Placement_Skill	28
5.5 Branch	28
5.6 Placement_Branch	29
5.7 Attachment.....	29
5.8 Admin	30
5.9 Student	30
5.10 Placement_Chunk	31
5.11 Entity – Relationship Diagram.....	32
6. SCREENSHOTS.....	33
7. CONCLUSION.....	34
8. FUTURE ENHANCEMENT.....	35
8.1 Automated Placement Email Ingestion.....	35
8.2 AWS Deployment.....	35

8.3 Stretch Goals	35
9. REFERENCES	36

LIST OF FIGURES

Fig. 2.1 Iterative Waterfall phases	10
Fig 4.1 High-Level Architecture.....	20
Fig 4.4 Use Case Diagram	21
Fig 4.3 Student Activity Diagram.....	22
Fig 4.4 Student Sequence Flow	23
Fig 4.5 Data Flow Diagram - Level 0.....	24
Fig 4.6 Data Flow Diagram - Level 1	25
Fig 5.11 Entity-Relationship Diagram.....	32

LIST OF TABLES

Table 2.1 Iterative Waterfall phases	11
Table 2.2 PlaceIntel Semester 5 schedule.....	11
Table 2.3 Risk assessment register	12
Table 2.4 Risk containment plan	13
Table 3.1 Technology responsibilities	14
Table 3.2 Fit Score inputs and behavior	16
Table 3.3 Chatbot retrieval and answer flow	17
Table 3.4 API endpoints	17
Table 5.1 Company Table.....	26
Table 5.2 Placement Table.....	27
Table 5.3 Skill Table.....	28
Table 5.4 Placement_Skill Table	28
Table 5.5 Branch Table.....	28
Table 5.6 Placement_Branch Table	29
Table 5.7 Attachment Table.....	29

Table 5.8 Admin Table	30
Table 5.9 Student Table	30
Table 5.10 Placement_Chunk Table	31

1. INTRODUCTION

1.1 Problem Summary

1.1.1 Problem Identification

At CHARUSAT, placement information is not difficult because it is unavailable; it is difficult because it is scattered. A student may find one company notice in a Google Groups email, another in a PDF attachment, and a third in a message forwarded by a class representative. After a few placement seasons, hundreds of messages accumulate. Even a basic question such as “Which companies visited for my branch last year?” becomes a manual search exercise. Questions about package ranges, recurring skills, deadlines, or the difference between two similar roles take even longer because there is no shared structure for comparing them.

The same problem appears on the administration side. A placement administrator has to type or reformat the same information for different notices, spreadsheets, or student requests. Small inconsistencies are easy to introduce: a company name may be written in two ways, a package may be stored without a clear minimum or maximum, or an eligibility condition may be present only in an attachment. Students then receive information that is technically present but not easy to use.

The aim is to turn placement history into a maintained information source that students can query and that an administrator can correct. The first version is intentionally based on manual entry. That lets us test whether the database structure, search behavior, and student-facing screens make sense with real notices instead of assuming that an automated importer will solve every problem for us.

1.1.2 Problem Solution

PlaceIntel provides one web platform for placement data at CHARUSAT. An administrator enters a company and its placement opportunities through the admin panel. Each placement can store the role, package range, deadline, CGPA requirement, description, eligible branches, required skills, and related attachment metadata. Once saved, the information is available through the student interface for searching, filtering, comparison, and analytics.

The student experience is meant to reduce reading and guessing. A student can narrow listings by branch, skill, package range, and other available fields, open a detailed placement record, and see a Fit Score calculated from the student profile. The same historical data also powers the chatbot. Placement descriptions are divided into chunks, embedded, stored in the pgvector extension inside the same PostgreSQL database, and retrieved when a question is asked. The language model receives the relevant chunks rather than the whole database and the answer includes the notice it came from. If retrieval does not produce useful evidence, the bot says that it does not know instead of filling the gap with an invented answer.

1.2 What's Different About This Project

1.2.1 The Fit Score

The Fit Score is the feature we expect students to notice first. Placement listings often contain enough detail to judge suitability, but a student has to read the fine print on every posting before making that judgement. In a semester with dozens of opportunities, that is a real time cost. We therefore compare the logged-in student's branch, CGPA, and self-listed skills with the requirements stored for a placement and show a compatibility percentage on the listing. The score is a guide for prioritising attention, not a promise that a student will be selected.

The score is personal rather than fixed. Two students can open the same placement and see different percentages because their profiles differ. Branch eligibility and CGPA are treated as strong signals, while skill overlap adds evidence about how closely the student's preparation matches the role. The calculation is kept explainable: the backend can return the component values so that the UI does not present a mysterious number without context.

1.2.2 Building the Chatbot

For PlaceIntel, asking a question of the placement history is central to the reason for organising the history in the first place. We therefore built a bounded chatbot: it answers from stored placement notices, identifies the source notice, and refuses to guess when the retrieved material is insufficient.

The chatbot is kept as a small Python FastAPI service called by the main backend over HTTP. That boundary gives the AI work its own place without mixing embedding and retrieval code into every placement route. It also makes the failure behavior clearer: if the service is unavailable, the application can report that the answer could not be generated rather than silently presenting an ungrounded response.

1.3 Purpose of This Document

This SRS is the working reference for the three-member PlaceIntel team and the basis on which the completed system can be reviewed. It records the problem, the architecture, the data and API contracts, the risks, and the boundaries kept for this semester. A faculty reviewer should be able to understand not only what the system contains, but why the team chose to build it in this order.

1.4 Scope and Boundaries

Its scope includes manual/admin data entry for roughly 50–100 placement notices during the semester, role-based login, placement search and filtering, company and placement management, analytics, Fit Score calculation, and the grounded chatbot. The local development environment is Docker and Docker Compose. The database is PostgreSQL 15 with pgvector enabled in the same database; there is no separate vector database service in this system.

An administrator enters the notice through the platform, and the application stores the structured record and any attachment metadata needed by the interface. The automatic collection, cloud hosting, or a second college are not part of the current implementation.

1.5 Stakeholders and Intended Users

- Students: search opportunities, inspect requirements, understand how a listing relates to their profile, review analytics, and ask questions about placement history.
- Placement administrator: enter and maintain company, placement, skill, branch, and attachment information; correct mistakes; and view system statistics.
- Project team: maintain the codebase, database schema, chatbot service, test data, and documentation as a three-person Semester 5 team.
- Faculty reviewer: evaluate whether the platform addresses the placement-information problem with a coherent, implemented design.

1.6 Assumptions and Dependencies

For now dataset is based on real CHARUSAT placement notices and is expected to contain roughly 50-100 manually entered records. The team assumes PostgreSQL 15, Docker, and Docker Compose are available on the development and evaluation machines. The chatbot depends on an LLM service for answer generation, but its answer is constrained by retrieved placement chunks and must report when the evidence is insufficient.

1.7 Document Overview

Chapter 2 explains the iterative plan and fourteen-week schedule. Chapter 3 describes the product layers, functions, API, Fit Score, chatbot, validation, and quality targets. Chapter 4 reserves labelled spaces for diagrams expected in a college SRS. Chapter 5 documents the relational schema and field-level data dictionary. Chapters 6 and 7 cover screens and conclusion, while Chapter 8 isolates future enhancement work and Chapter 9 lists references.

2. PLANNING

2.1 Model Description — Iterative Waterfall Model

We follow an Iterative Waterfall Model because the project needs a clear sequence for a college submission but still benefits from short feedback loops. The work moves through feasibility study, requirements, design, coding and unit testing, integration and system testing, and maintenance.

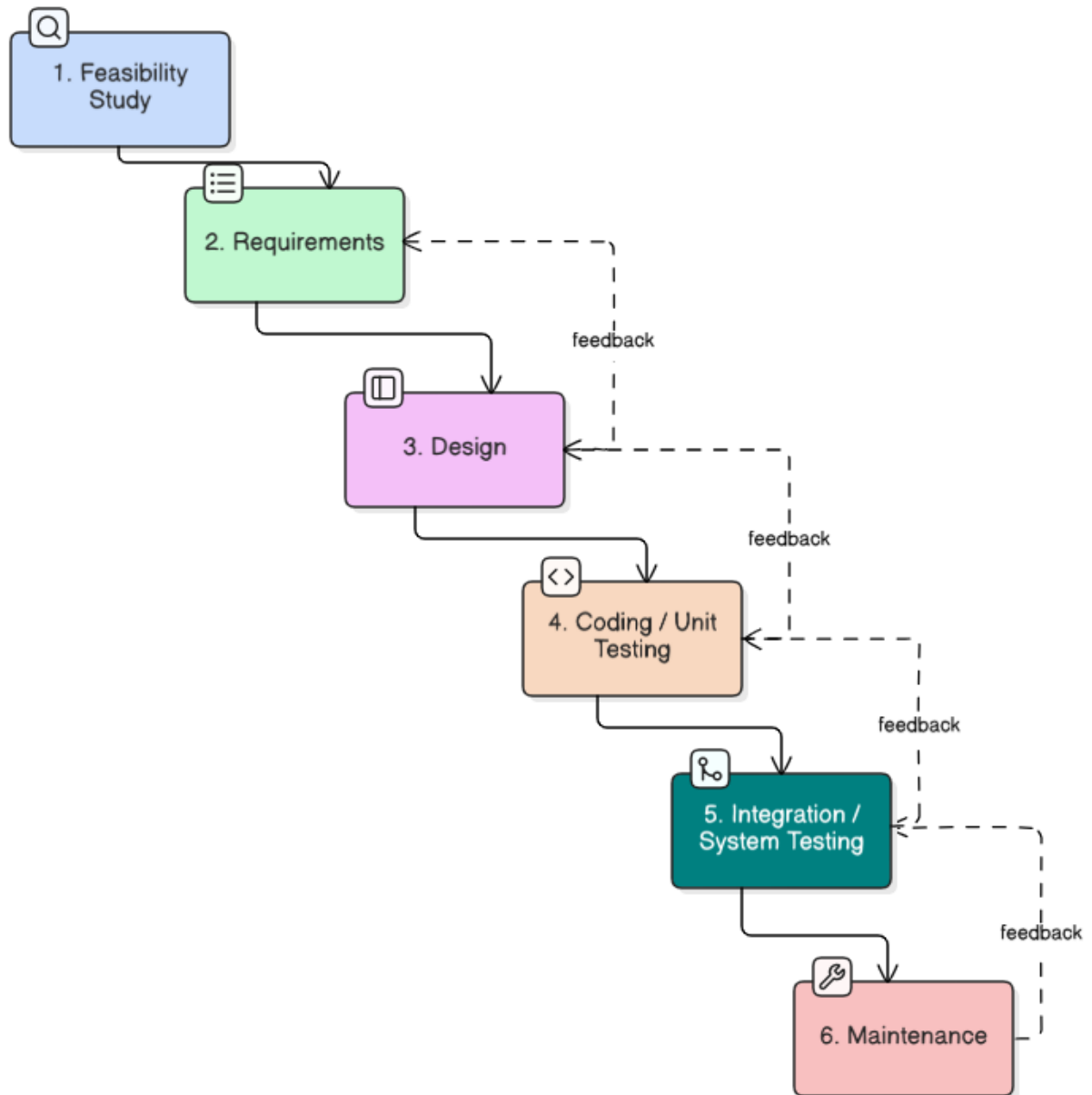


Fig. 2.1 Iterative Waterfall phases

Table 2.1 Iterative Waterfall phases

Phase	How we use it for PlaceIntel
Feasibility study	Check that a three-person team can build the core platform locally with the selected stack and a manageable manually entered dataset.
Requirements	Agree on student and admin workflows, Fit Score behavior, chatbot grounding behavior, and data fields.
Design	Define the normalized PostgreSQL schema, API contracts, role permissions, page responsibilities, and the boundary between the backend and FastAPI chatbot service.
Coding / unit testing	Implement each module and test its local behavior, including validation, authentication middleware, filtering, scoring, and retrieval helpers.
Integration / system testing	Run the frontend, REST API, database, and chatbot service together with real placement records and check complete user journeys.
Maintenance	Fix defects found during demonstration and documentation review.

2.2 Our Schedule

Table 2.2 PlaceIntel Semester 5 schedule

Weeks	Planned work	Expected result
Week 1–2	Finalize scope, divide ownership, set up Git repository and Docker environment.	A runnable skeleton and agreed boundaries.
Week 3–4	Design the database schema; run PostgreSQL 15 and Prisma models.	Initial migrations and verified relations.
Week 5–6	Build backend APIs for companies, placements, skills, branches, and login while the frontend builds search, listing, and detail pages in parallel.	First end-to-end CRUD and browsing path.
Week 7	Start entering real placement notices by hand. Use the records to expose schema and validation problems.	A realistic sample dataset and schema feedback.
Week 8	Build admin panel and CRUD screens; wire up authentication properly.	Protected admin workflow.

Week 9	Build analytics dashboard and Fit Score feature.	Charts and personalised percentage badges.
Week 10–11	Build chatbot: chunk notice text, generate embeddings, search pgvector, connect to LLM, and add basic chat UI.	Grounded chatbot path with source notice.
Week 12	Integrate all services and fix issues between modules.	Complete workflow.
Week 13	Polish screens, fix bugs, and write documentation.	Submission-ready application and report draft.
Week 14	Prepare the demo and final submission.	Demonstration script, final checks, and handover.

2.3 Risk Management

2.3.1 Risk Identification

The main project risk is coordination. Three people are sharing coursework and each owns a different part of the system, so an API or schema change can affect someone else's page. A technical risk is that no team member has used pgvector before; the chatbot depends on getting embeddings, vector storage, and similarity search to work together. These are not abstract risks for us; they are the kinds of issues that can make a technically correct project fail during a demo or fail to be useful.

Table 2.3 Risk assessment register

Risk type	Specific risk	Probability	Impact	Priority = $P \times I$
Project	Three members juggle coursework and a delay in one module blocks integration.	3/5	4/5	12
Project	Manual entry of 50–100 notices takes longer than expected.	4/5	3/5	12
Technical	The team is unfamiliar with pgvector and retrieval quality is poor.	3/5	5/5	15

Technical	Authentication or role checks are added late and require rework.	2/5	5/5	10
Technical	Real notice text has inconsistent package, CGPA, or skill formats.	4/5	4/5	16

2.3.2 Risk Containment

Table 2.4 Risk containment plan

Risk	Containment approach	Risk leverage
Team availability	Reduce: assign ownership, commit small changes regularly, and agree on API/schema changes before implementation.	A shared repository and short integration checkpoints lower coordination cost.
Manual entry workload	Reduce: split notices between members, define mandatory fields, and validate records as they are entered instead of doing one large cleanup at the end.	A clean sample dataset also tests the schema.
pgvector unfamiliarity	Reduce: build a small retrieval experiment before connecting the UI; store source placement IDs with chunks and inspect returned text.	Early technical learning protects the Week 11 integration deadline.
Inconsistent notice formats	Reduce: keep raw description text, use nullable fields where a notice genuinely omits data, and show unknown values honestly.	The system does not invent a package or eligibility rule.
Clunky student experience	Avoid/reduce: test the search and detail flow with classmates, keep filters understandable, and show the Fit Score with a short explanation.	A short feedback loop links adoption to visible UI changes.
Score misunderstanding	Reduce: label it as compatibility, expose the contributing factors, and state that final selection remains with the company and placement process.	Clear wording prevents a useful guide from becoming a false promise.

3. DETAIL DESCRIPTION

3.1 System Overview

PlaceIntel is divided into a React/TypeScript frontend, a Node.js/Express/TypeScript REST API, a PostgreSQL 15 database accessed through Prisma, and a small Python FastAPI chatbot service. The browser talks to the main API. The API owns authentication, validation, placement queries, Fit Score calculation, and the decision to call the chatbot. The chatbot service owns chunk retrieval and the LLM request; it does not become a second source of placement data.

Table 3.1 Technology responsibilities

Layer	Responsibility	Reason for the choice
React + TypeScript + Tailwind CSS	Pages for login, search, placement details, analytics, admin CRUD, and chat.	Component structure and typed data make it easier for the frontend member to work against API contracts.
Node.js + Express + TypeScript	REST routes, validation, JWT middleware, business logic, Fit Score, and service-to-service call.	One typed service coordinates the main application workflow.
Prisma ORM	Schema models, migrations, and parameterised access to PostgreSQL.	The relational design is easier to inspect and change through typed models.
PostgreSQL 15 + pgvector	Structured placement data and vector embeddings in the same database.	Companies, placements, branches, and skills are relational; keeping chunks beside their source records simplifies traceability.
Python FastAPI chatbot service	Embedding/retrieval flow and grounded LLM answer generation.	The AI experiment stays isolated from the core API without requiring a separate data store.
Docker Compose	Local orchestration for the application services and database.	Every team member can start the same development environment.

3.2 Functional Areas

3.2.1 Authentication and Roles

The login endpoint accepts the credentials required for a student or administrator and returns a JWT after verification. Passwords are stored as bcrypt hashes, never as plain text. The token identifies the authenticated user and role so the backend can protect administrative routes. A student can browse and use student features; only an administrator can create, update, or delete company and placement records through the admin panel.

Authentication is checked on the server even when the interface hides a button. This matters because a hidden “Delete” button is a user-interface decision, not a security boundary. Invalid credentials and expired or malformed tokens return a clear error response so the frontend can ask the user to log in again.

3.2.2 Placement Search and Filters

The listing page is the main student entry point. It supports filters for skill, package range, branch, and year, with the backend applying the filters to structured records. The search result should show the company, position, package range when available, deadline, eligible branches, and Fit Score badge for the logged-in student. Opening a result leads to a detail view rather than forcing a student to reconstruct the notice from several screens.

Filtering is performed using database fields and relationships rather than by searching a single long text blob. That distinction is important: a branch filter should use the `Placement_Branch` relationship, while a skill filter should use `Placement_Skill`. The original description remains available for the details that cannot be safely reduced to a field.

3.2.3 Fit Score

The Fit Score endpoint compares the student profile with one placement. The calculation uses branch alignment, CGPA alignment, and skill overlap. A placement with a branch mismatch should not receive the same score as one that accepts the student’s branch; similarly, the student’s CGPA should be compared with the stored requirement when a requirement exists. Skill overlap adds a useful signal but does not replace eligibility conditions.

Table 3.2 Fit Score inputs and behavior

Input	How it is used	Visible behavior
Branch	Check whether the student branch appears among the placement's eligible branches.	The score reflects a branch match; the detail page still shows the actual branch rule.
CGPA	Compare the student CGPA with CGPA_Req when the placement provides one.	The student can see whether the minimum is met rather than relying only on the percentage.
Skills	Compare the student's self-listed skills with Placement_Skill entries.	Matching and missing skills can be explained beside the badge.
Missing requirement	Do not treat absent data as a match. Preserve "not specified" where the notice has no value.	The score remains a guide and does not manufacture eligibility.

3.2.4 Analytics Dashboard

The analytics dashboard turns the stored records into views that are difficult to produce from old messages. It can show company and placement counts, package ranges, skill demand, branch distribution, and year-wise or other available trends. Chart.js or Recharts renders the charts in the frontend, while the `/api/stats` endpoint supplies aggregated values. The dashboard is descriptive: it reports what exists in the entered dataset and does not claim to predict future hiring.

3.2.5 AI Chatbot

When placement text is prepared for the chatbot, it is broken into manageable chunks. Each chunk is converted into an embedding and stored in Placement_Chunk with its source Placement_ID. A student question is embedded or otherwise prepared for similarity search. The chatbot service retrieves the most relevant chunks from pgvector and passes those chunks, together with the question, to an LLM. The answer is returned with the source notice so the student can connect the response to a real record.

Table 3.3 Chatbot retrieval and answer flow

Step	System action	Guardrail
1. Receive	Main API accepts `{ question }` from the student UI.	Require an authenticated student request and reject empty questions.
2. Retrieve	FastAPI service searches the pgvector embeddings in PostgreSQL.	Keep the source placement ID and notice name with each result.
3. Generate	LLM writes a concise answer from the retrieved chunks.	Prompt the model not to add facts absent from the supplied context.
4. Return	Main API returns `{ answer, source_notice }` to the UI.	Display the source and an uncertainty message when evidence is weak.
5. Fail safely	If no relevant chunk is found or the service fails, report that the system cannot answer from the available placement data.	Do not substitute a guessed answer.

3.3 API Endpoint Summary

Table 3.4 API endpoints

Endpoint	Method	Purpose	Access
/api/login	POST	Verify credentials and issue a JWT.	Student/Admin
/api/companies	GET, POST, PUT, DELETE	List and maintain company records.	GET: authenticated; writes: Admin
/api/placements	GET, POST, PUT, DELETE	Search, create, update, and delete placement records. GET accepts skill, package range, branch, and year filters.	GET: authenticated; writes: Admin

/api/skills	GET	Return the skills available for filters and admin forms.	Authenticated
/api/stats	GET	Return values for the analytics dashboard.	Authenticated
/api/placements/{id}/fit-score	GET	Calculate the current student's compatibility percentage for a listing.	Student
/api/chat	POST	Accept `{question}` and return `{answer, source_notice}` from grounded retrieval.	Student

3.4 Validation and Error Handling

The admin form validates required identifiers and relationships before a record is saved. A placement must point to a real company; mapping rows must point to existing placements, skills, or branches; package and CGPA values must be numeric when provided; and a deadline must be a valid date. The application keeps the full description because it is safer to preserve an unusual notice condition than to force every sentence into a field that was never designed for it.

The API returns structured error responses for invalid input, missing records, authentication failures, and service errors. The frontend converts those responses into messages a student or administrator can act on. A database failure is not presented as an empty search result, because “there are no placements” and “the request could not be completed” mean different things to the user.

3.5 Non-Functional Targets

Filtered placement requests should normally return within two seconds for the expected 50-100 notice dataset. A first-time administrator should be able to create a placement after a short walkthrough without consulting the database directly. Passwords must be bcrypt hashes, protected routes must validate JWTs, and admin actions must be role-checked on the server. Validation and service failures must return structured errors instead of empty or misleading success responses. Chatbot changes must not require rewriting placement search or analytics routes. The complete Semester 5 stack should start with Docker Compose on the team's development machines.

3.6 Practical Acceptance Checks

The team will demonstrate the system with the following checks: an administrator can log in and add a company; the administrator can add a placement with skills and branches; a student can filter the listing by branch and skill; the detail page shows the stored eligibility and package values; two different student profiles can receive different Fit Scores for the same listing; the dashboard reflects the entered records; and the chatbot either cites a relevant notice or clearly states that it cannot answer from the available data.

4. DIAGRAMS

4.1 High Level Architecture

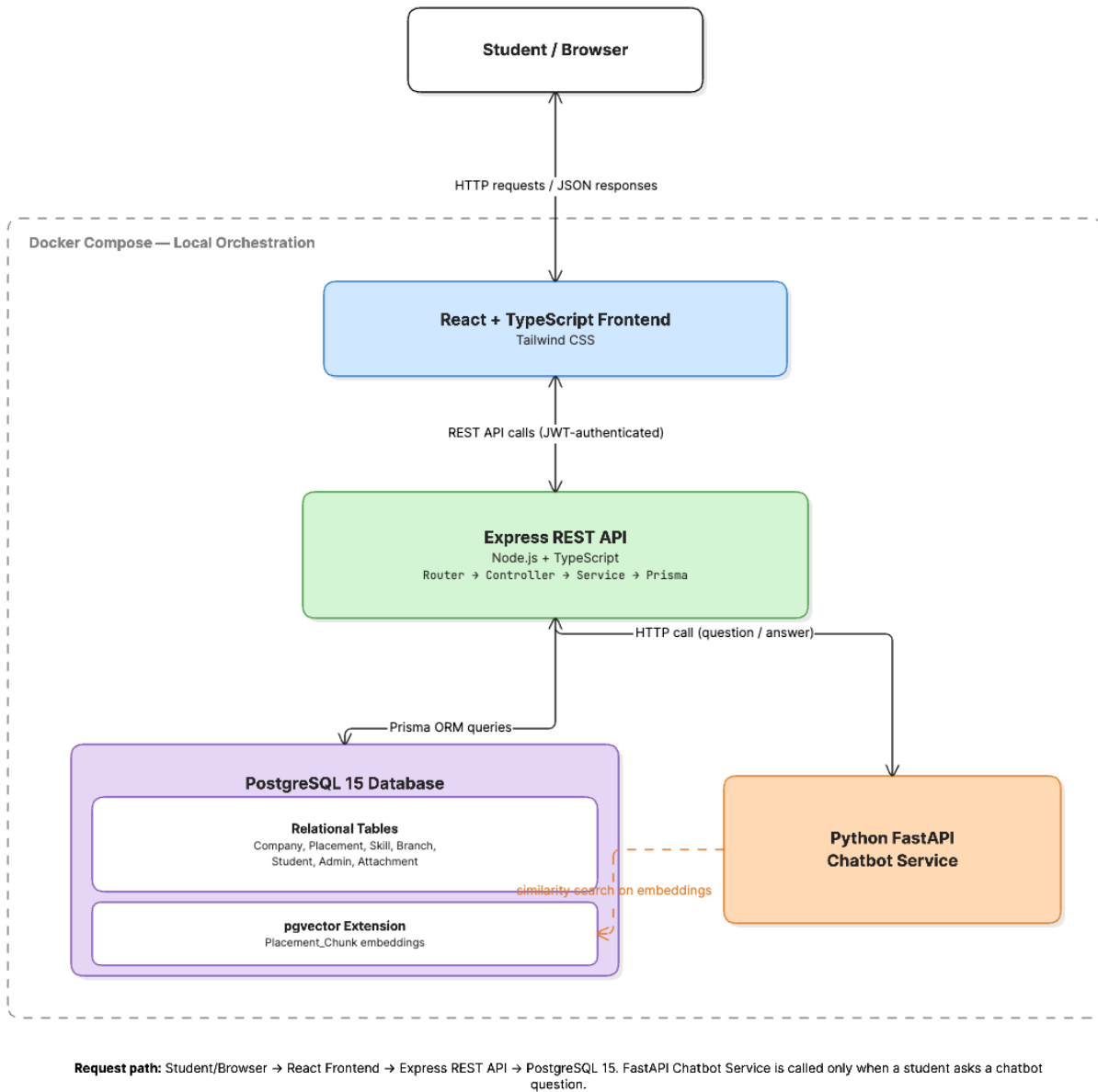


Fig 4.1 High-Level Architecture

4.2 Use Case Diagram

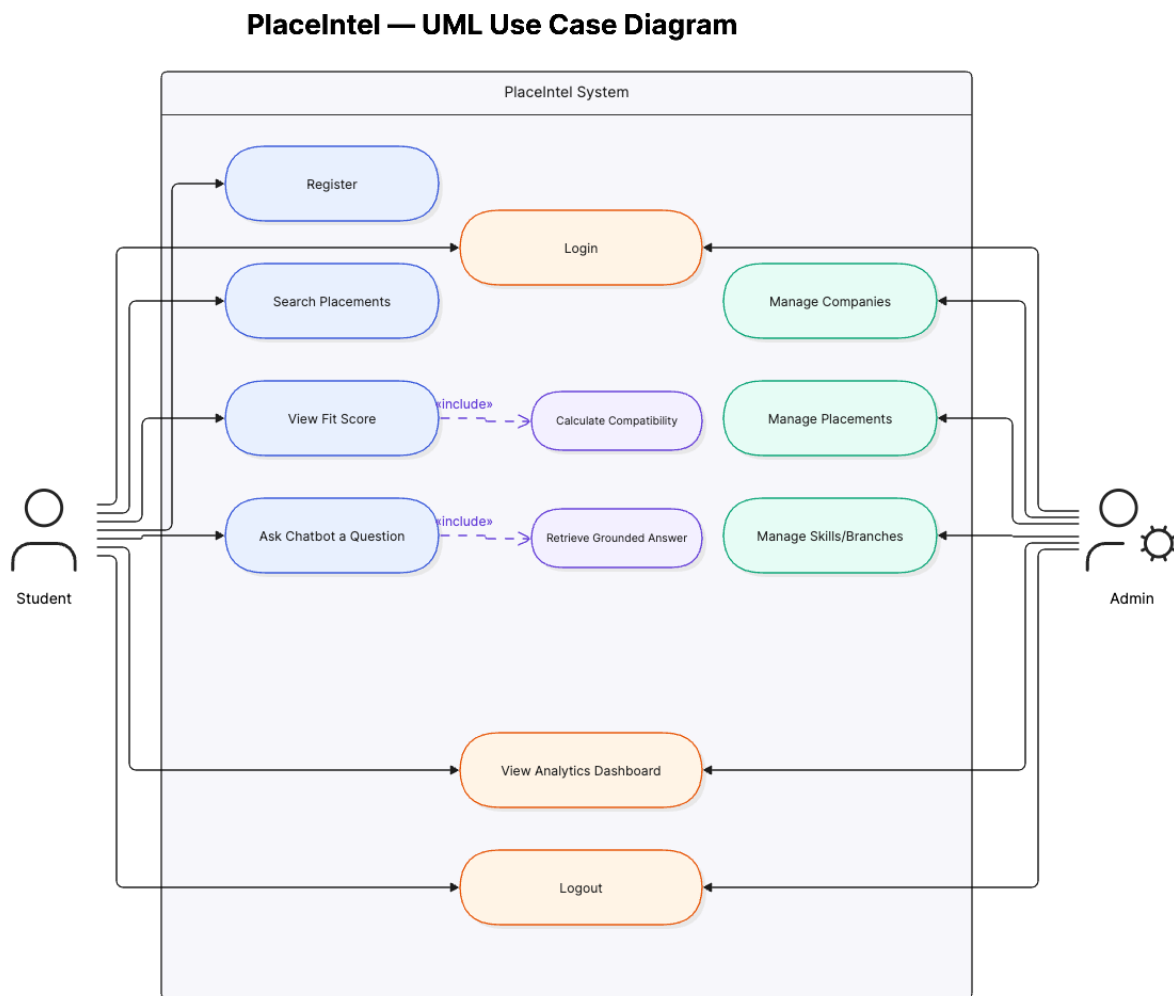


Fig 4.4 Use Case Diagram

4.3 Student Activity Diagram

PlaceIntel — Student Flow (UML Activity Diagram)

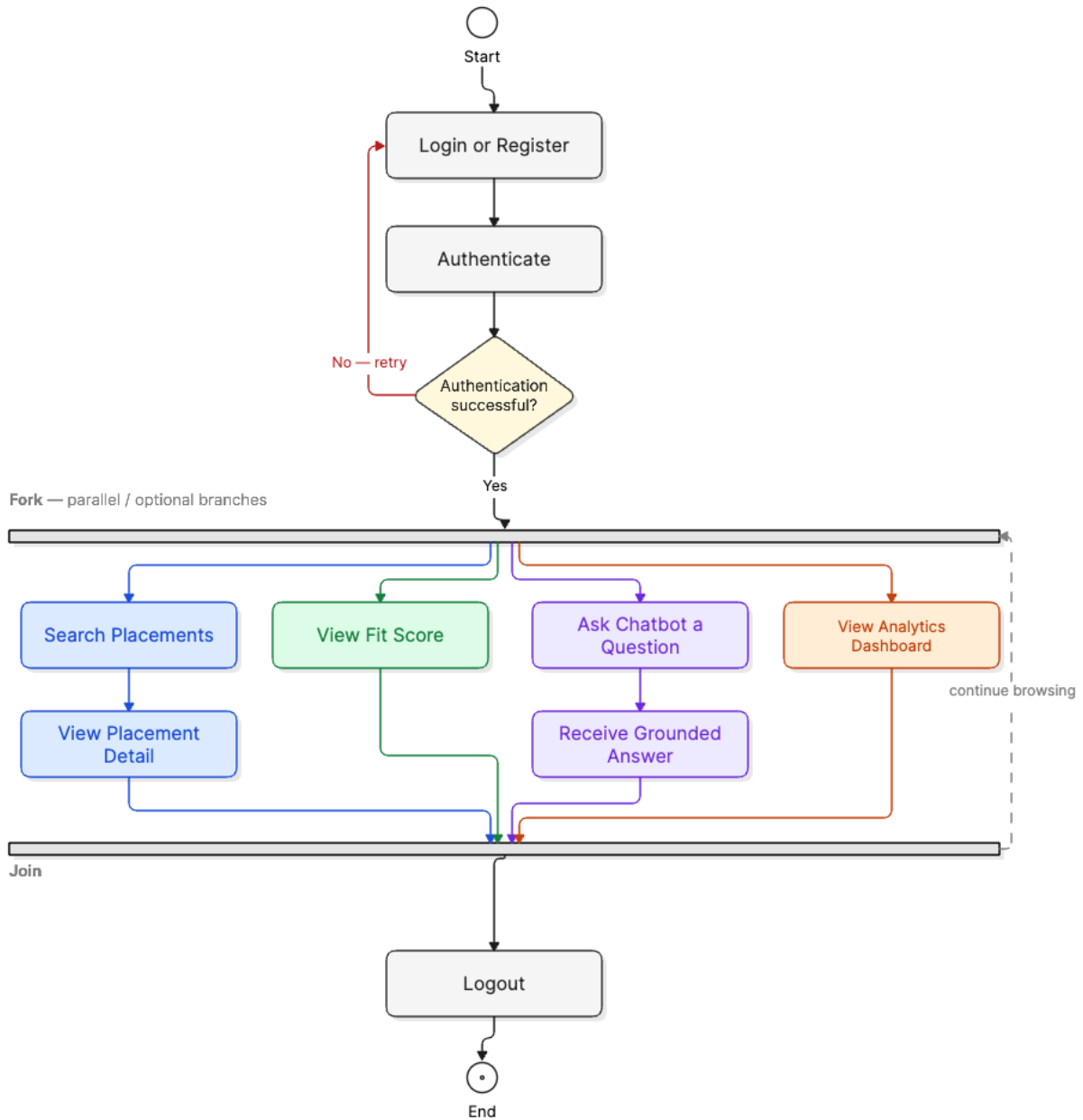


Fig 4.3 Student Activity Diagram

4.4 Studen Sequence Flow Diagram

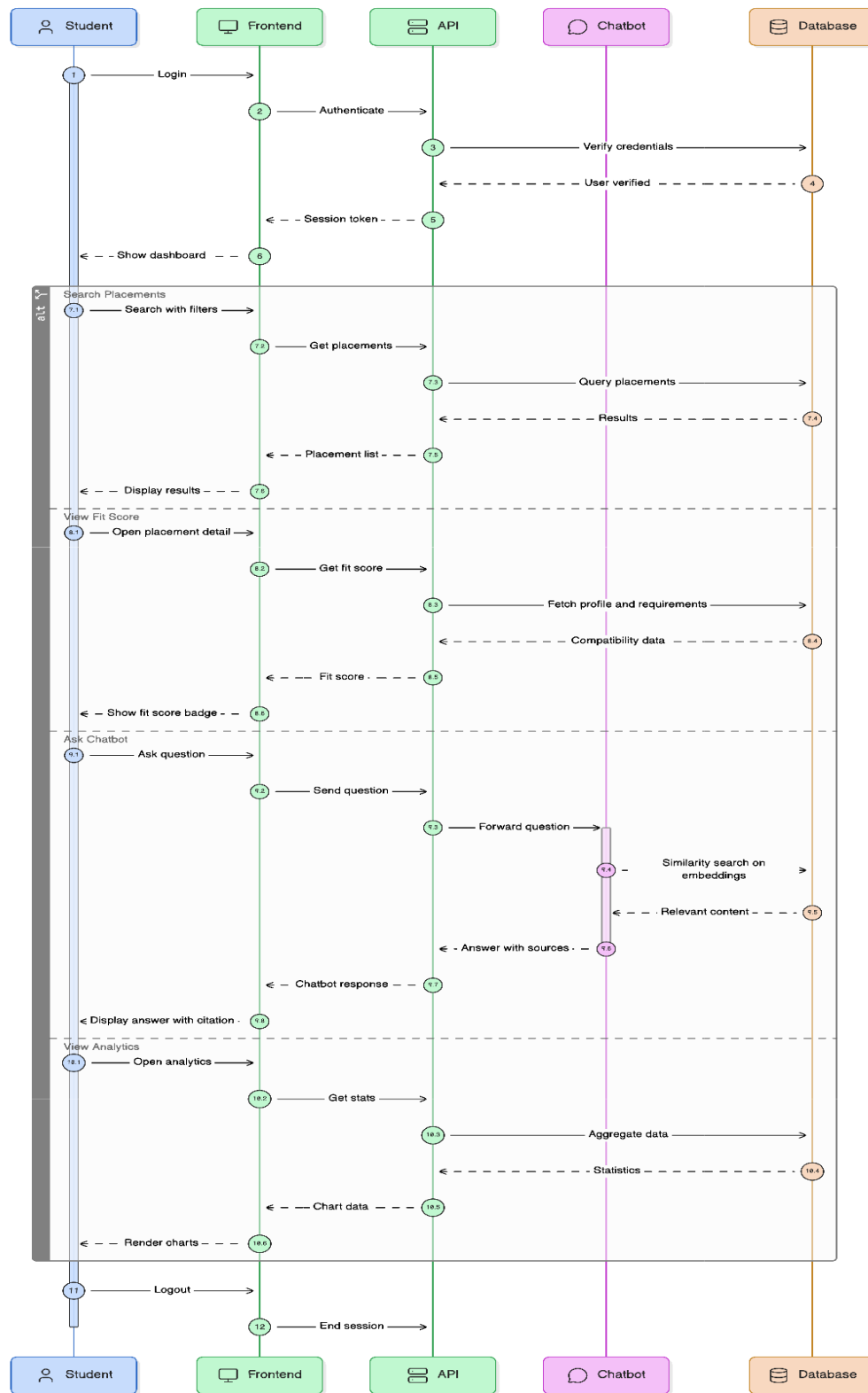


Fig 4.4 Student Sequence Flow

4.5 Data Flow Diagram Level – 0

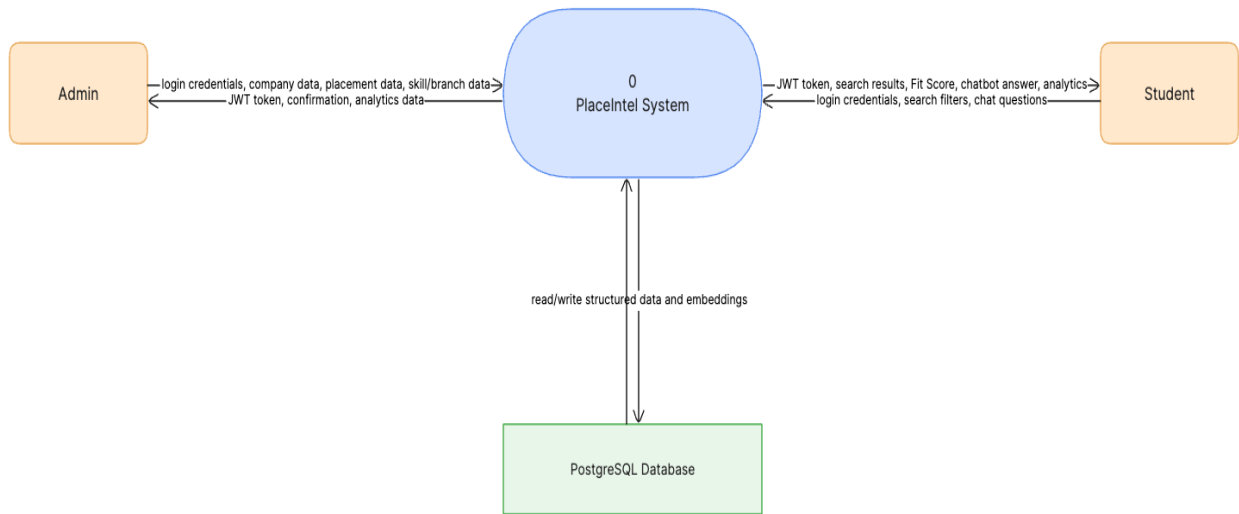


Fig 4.5 Data Flow Diagram - Level 0

4.6 Data Flow Diagram Level – 1

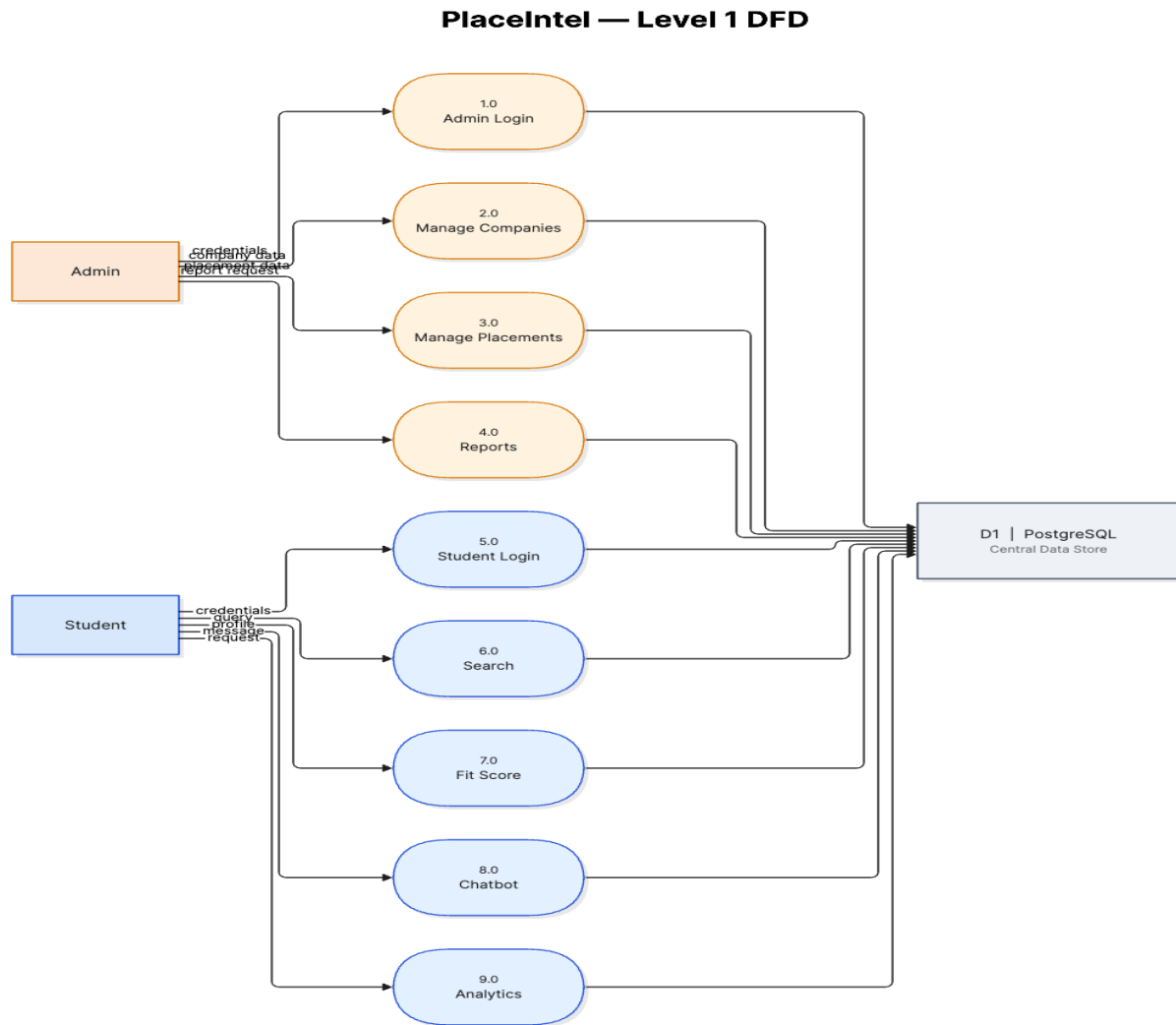


Fig 4.6 Data Flow Diagram - Level 1

5. DATA DICTIONARY

The following tables describe the schema exactly as used by the project. Primary keys are marked PK, foreign keys are marked FK, and composite mappings use both columns as their key. Sizes are implementation guidance rather than a reason to truncate real notice text.

5.1 Company

Table 5.1 Company Table

Field Name	Datatype	Size	Constraint	Reference	Description
Company_ID	Integer	—	PK	—	Unique company identifier
Company_Name	Varchar	150	Required	—	Name displayed in company and placement views
Website	Varchar	255	Nullable	—	Official or reference website
Industry	Varchar	100	Nullable	—	Industry category used for display and analysis

5.2 Placement

Table 5.2 Placement Table

Field Name	Datatype	Size	Constraint	Reference	Description
Placement_ID	Integer	—	PK	—	Unique placement identifier
Company_ID	Integer	—	Required, FK	Company.Company_ID	Company offering the placement
Position	Varchar	150	Required	—	Role or position title
Min_Package	Decimal	10,2	Nullable	—	Lower end of package range
Max_Package	Decimal	10,2	Nullable	—	Upper end of package range
Deadline	Date	—	Nullable	—	Last date to apply, when known
CGPA_Req	Decimal	3,2	Nullable	—	Minimum CGPA, when stated
Description	Text	Large	Required	—	Full notice and additional conditions

5.3 Skill

Table 5.3 Skill Table

Field Name	Datatype	Size	Constraint	Reference	Description
Skill_ID	Integer	—	PK	—	Unique skill identifier
Skill_Name	Varchar	100	Required, unique	—	Controlled skill label

5.4 Placement_Skill

Table 5.4 Placement_Skill Table

Field Name	Datatype	Size	Constraint	Reference	Description
Placement_ID	Integer	—	PK, FK	Placement.Placement_ID	Placement side of composite mapping
Skill_ID	Integer	—	PK, FK	Skill.Skill_ID	Skill side of composite mapping

5.5 Branch

Table 5.5 Branch Table

Field Name	Datatype	Size	Constraint	Reference	Description
Branch_ID	Integer	—	PK	—	Unique branch identifier

Branch_Name	Varchar	150	Required, unique	—	Academic branch name
-------------	---------	-----	---------------------	---	----------------------------

5.6 Placement_Branch

Table 5.6 Placement_Branch Table

Field Name	Datatype	Size	Constraint	Reference	Description
Placement_ID	Integer	—	PK, FK	Placement.Placement_ID	Placement side of composite mapping
Branch_ID	Integer	—	PK, FK	Branch.Branch_ID	Eligible branch side of mapping

5.7 Attachment

Table 5.7 Attachment Table

Field Name	Datatype	Size	Constraint	Reference	Description
Attachment_ID	Integer	—	PK	—	Unique attachment identifier
Placement_ID	Integer	—	Required, FK	Placement.Placement_ID	Placement whose notice is supported
File_Path	Varchar	500	Required	—	Path or served location of the file

File_Type	Varchar	50	Required	—	File extension or MIME category
-----------	---------	----	----------	---	---------------------------------

5.8 Admin

Table 5.8 Admin Table

Field Name	Datatype	Size	Constraint	Reference	Description
Admin_ID	Integer	—	PK	—	Unique administrator identifier
Admin_Username	Varchar	100	Required, unique	—	Administrator login name
Admin_Password_Hash	Varchar	255	Required	—	bcrypt hash, never a plain password
Admin_Email	Varchar	255	Required	—	Administrator contact/login email

5.9 Student

Table 5.9 Student Table

Field Name	Datatype	Size	Constraint	Reference	Description
Student_ID	Integer	—	PK	—	Unique student identifier

Student_Name	Varchar	150	Required	—	Student display name
Student_Email	Varchar	255	Required, unique	—	Student login/contact email
Password_Hash	Varchar	255	Required	—	Protected password hash
Branch_ID	Integer	—	Required, FK	Branch.Branch_ID	Student academic branch
CGPA	Decimal	3,2	Required	—	CGPA used in Fit Score
Skills	Text / linked list	Variable	Nullable	Skill or skill-list representation	Self-listed skills used in Fit Score

5.10 Placement_Chunk

Table 5.10 Placement_Chunk Table

Field Name	Datatype	Size	Constraint	Reference	Description
Chunk_ID	Integer	—	PK	—	Unique chunk identifier
Placement_ID	Integer	—	Nullable, FK	Placement.Placement_ID	Source placement when resolved
Chunk_Text	Text	Large	Required	—	Text fragment used as

					retrieval evidence
Embedding	Vector	Configure d dimension	Required	pgvector	Embedding used for similarity search

5.11 Entity – Relationship Diagram

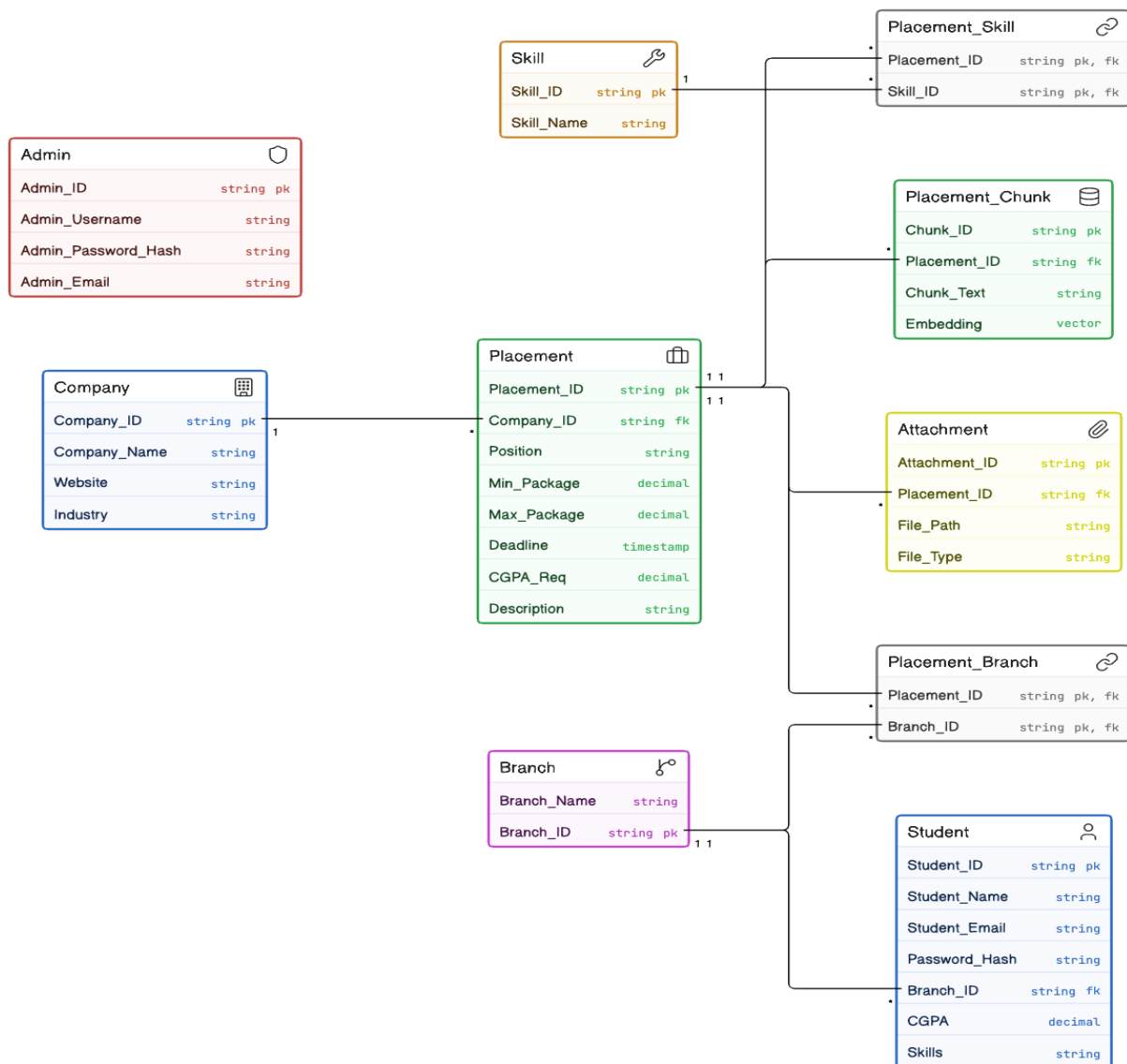


Fig 5.11 Entity-Relationship Diagram

6. SCREENSHOTS

We will add Screenshots when the UI design will be ready.

7. CONCLUSION

PlaceIntel addresses a small but persistent problem at CHARUSAT: placement information exists in many places, while students need one place where they can search, compare, and ask questions. Our system makes an administrator enter the data once and then exposes that data through listings, filters, analytics, a personal Fit Score, and a source-aware chatbot.

We had to decide which facts deserved their own database fields, how to keep many-to-many branch and skill relationships clean, how to make a percentage understandable, and how to stop the chatbot from sounding confident when the data does not support an answer. Building with three people also made boundaries important: the frontend, main backend, database, and chatbot service needed contracts that could be tested together.

The main takeaway of the project is that placement history becomes more useful when it is treated as maintained information rather than as a pile of notices. The current system is deliberately grounded in the data entered. That gives us a working platform to demonstrate, test, and improve before adding larger sources or deployment environments.

8. FUTURE ENHANCEMENT

8.1 Automated Placement Email Ingestion

The first Semester 6 enhancement is automatic fetching of new placement messages from Gmail or Google Groups instead of requiring an administrator to type each notice by hand. The importer would need to recognise duplicate and forwarded messages, extract structured fields from email text, and pull information out of attached PDFs and images. It would also need a review path because a parser should not publish an uncertain package or eligibility value as if it were confirmed.

We intentionally waited on this work. Manual entry in Semester 5 proves whether the database structure can hold real notices and whether the student screens actually use the fields we designed. Starting with automation before that proof would make it harder to tell whether an error came from the schema or from the importer. Gmail/Google API access and the variety of attachment formats are also their own project-sized tasks; they should not block the core platform and chatbot.

8.2 AWS Deployment

After ingestion is stable locally, the platform can move from Docker Compose to AWS. The deployment work would include selecting managed hosting for the API and frontend, a managed PostgreSQL option, secure storage for attachments, secrets management, logging, backups, and a repeatable release process. The application behavior should remain the same while the hosting environment changes.

8.3 Stretch Goals

- Amazon Cognito for managed student and administrator login if the authentication requirements justify the migration.
- Alerts for students when a new placement matches their branch or profile.
- Groundwork for reuse by other colleges, including separating CHARUSAT-specific labels and policies from the core data model.
- Terraform so the AWS environment can be recreated consistently rather than configured by hand.

9. REFERENCES

1. React & TypeScript Documentation — <https://react.dev/> and <https://www.typescriptlang.org/docs/>
2. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
3. Express.js Documentation — <https://expressjs.com/>
4. Prisma ORM Documentation — <https://www.prisma.io/docs>
5. PostgreSQL 15 Documentation — <https://www.postgresql.org/docs/15/>
6. pgvector Extension Documentation — <https://github.com/pgvector/pgvector>
7. FastAPI Documentation — <https://fastapi.tiangolo.com/>
8. Chart.js / Recharts Documentation — <https://www.chartjs.org/docs/> and <https://recharts.org/>
9. Docker Compose Documentation — <https://docs.docker.com/compose/>
10. Mermaid Documentation — <https://mermaid.js.org/>
11. CHARUSAT Placement Notices — Google Groups emails, PDFs, and forwarded messages used as the Semester 5 data-entry source.